electronics

MDPI

Article

A Systematic Study of Adversarial Attacks Against Network Intrusion Detection Systems

Sanidhya Sharma and Zesheng Chen *

College of Engineering, Technology, and Computer Science, Purdue University Fort Wayne, Fort Wayne, IN 46805, USA; shars05@pfw.edu

* Correspondence: chenz@pfw.edu

Abstract: Network Intrusion Detection Systems (NIDSs) are vital for safeguarding Internet of Things (IoT) networks from malicious attacks. Modern NIDSs utilize Machine Learning (ML) techniques to combat evolving threats. This study systematically examined adversarial attacks originating from the image domain against ML-based NIDSs, while incorporating a diverse selection of ML models. Specifically, we evaluated both white-box and black-box attacks on nine commonly used ML-based NIDS models. We analyzed the Projected Gradient Descent (PGD) attack, which uses gradient descent on input features, transfer attacks, the score-based Zeroth-Order Optimization (ZOO) attack, and two decision-based attacks: Boundary and HopSkipJump. Using the NSL-KDD dataset, we assessed the accuracy of the ML models under attack and the success rate of the adversarial attacks. Our findings revealed that the black-box decision-based attacks were highly effective against most of the ML models, achieving an attack success rate exceeding 86% across eight models. Additionally, while the Logistic Regression and Multilayer Perceptron models were highly susceptible to all the attacks studied, the instance-based ML models, such as KNN and Label Spreading, exhibited resistance to these attacks. These insights will contribute to the development of more robust NIDSs against adversarial attacks in IoT environments.

Keywords: machine learning; adversarial attacks; network intrusion detection systems

check for updates

Citation: Sharma, S.; Chen, Z. A Systematic Study of Adversarial Attacks Against Network Intrusion Detection Systems. *Electronics* **2024**, *13*, 5030. https://doi.org/10.3390/electronics13245030

Academic Editors: Ning Yang and Abdur R. Shahid

Received: 21 October 2024
Revised: 18 December 2024
Accepted: 19 December 2024
Published: 21 December 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In today's digital age, cybersecurity is of paramount importance. Massive network infrastructures form the backbone of online communication or Internet of Things (IoT) networks, and their availability and proper functioning are critical. Major sectors such as the industrial IoT, social media, finance, healthcare, transportation, and administration heavily rely on these networks. For example, IoT networks in healthcare enable remote patient monitoring and real-time diagnostics, significantly improving patient outcomes. Similarly, in the financial sector, IoT networks enhance security and efficiency by streamlining processes such as automated transactions and fraud detection. In home security, IoT applications empower users with remote monitoring through smart cameras, door sensors, motion detectors, and alarm systems. When such networks are compromised, even briefly, the consequences can be catastrophic, impacting both the economy and people's lives. Malicious attackers often target these networks for personal gain, making it imperative to defend against such threats. Failing to do so can lead to significant harm to finances, health, and overall well-being.

One key defense mechanism for detecting incoming attacks is Network Intrusion Detection Systems (NIDSs) [1]. These systems monitor networks and generate alerts when suspicious activity is detected. Traditionally, NIDSs relied on signature-based detection to identify attacks. However, as networks expanded and the number of attacks increased, the number of rules or signatures in NIDSs became insufficient. Each new attack required the addition of a new rule or signature, making this approach increasingly impractical. Over time, this method was replaced by anomaly-based detection [2,3].

Modern NIDSs leverage Machine Learning (ML) to classify incoming packets as benign or malicious by learning the network's normal behavior and raising alerts for activities that deviate from this baseline. This approach is particularly effective in detecting novel attacks. Specifically, signature-based detection identifies attacks by matching their signatures against a predefined database. However, any attack not present in the database is classified as novel and can evade detection, even if it is only a slight variation of a known attack. The advent of Machine Learning has transformed modern NIDSs, enabling them to potentially detect these novel attacks, including those with subtly altered signatures.

However, the adoption of ML-based NIDSs has introduced a new threat: exploiting the vulnerabilities inherent in ML models. Specifically, adversarial attacks can deceive these models by introducing subtle changes to the input that are imperceptible to humans but cause the target Machine Learning model to misclassify it [4–11].

Recently, significant advancements have been made in the field of adversarial attacks, especially in image-classification tasks. A slight alteration to just a few pixels in an image, imperceptible to the human eye, can easily deceive a state-of-the-art Deep Learning (DL) model, causing it to misclassify the image [12]. These adversarial attacks present serious risks; for example, a self-driving car could be tricked into misreading a speed limit or street sign, potentially leading to dangerous accidents [13].

In the context of NIDSs, a similar strategy can be employed to cause an ML-based NIDS to misclassify a malicious input as benign, thereby allowing it to bypass network defenses. This tactic is commonly referred to as evasion [4]. However, input features for ML-based NIDSs differ fundamentally from those in image-based systems. While images consist of high-dimensional features, NIDS input features are typically low-dimensional. Additionally, DL models are the default in the image domain, whereas a variety of ML models, such as random forests or SVMs, can be applied in NIDSs. As a result, findings from the image domain may not be directly transferable to NIDSs. In this work, we aimed to address the following key questions:

- How can different adversarial attacks from the image domain be adapted to target ML-based NIDSs, and how do they perform?
- For a given ML model in NIDSs, such as random forest, to which adversarial attacks is the model vulnerable or resistant?

To address these questions, we developed nine NIDS models, using a variety of ML algorithms, including Logistic Regression (LGR), Multilayer Perceptron (MLP), SVM with linear and RBF kernels, Decision Tree Classifier (DTC), Random Forest Classifier (RFC), XGBoost, K-Nearest Neighbors (KNN), and Label Spreading (LS) [14]. These models were selected for this study because they are among the most popular ML methods and are widely used across various applications.

We also investigated both white-box and black-box adversarial attacks against NIDSs. In a white-box attack, the attacker possesses complete knowledge of the ML model. In contrast, a black-box attack assumes that the attacker has no knowledge of the model's structure or weights and can only access the model's output through queries. For white-box attacks, we focused on the Projected Gradient Descent (PGD) attack [15], a state-of-the-art method across various domains. Since this attack requires access to the gradient of the ML model with respect to the input features, we applied it only to the LGR and MLP models.

For black-box attacks, we explored three different approaches: transfer attacks, score-based attacks, and decision-based attacks. These three approaches have been extensively studied and widely applied in the image domain. In the transfer attacks, the adversarial examples generated using the PGD attack on the LGR or MLP models were applied to the other ML models. Additionally, we introduced a variant of the transfer attack, called PGD-assisted transfer attacks, in which the target ML model was queried during the generation of adversarial examples based on the PGD attack. For score-based attacks, we evaluated the Zeroth-Order Optimization (ZOO) attack [16]. For decision-based attacks, we examined the Boundary attack [17] and the HopSkipJump attack [18]. Since the gradients of the target models are not directly accessible in these three attacks, they estimate gradients by querying

the model multiple times. Specifically, the ZOO attack follows the Carlini and Wagner (CW) attack [19], using the L_∞ norm, but it replaces the gradient estimation process with finite differences calculated from model outputs. On the other hand, the Boundary and HopSkipJump attacks adopt a different approach, searching along the decision boundary by generating random examples from the target class and then searching between the original and the random example.

Through extensive experiments using the NSL-KDD dataset [20], we found that the PGD attack is a powerful white-box attack, capable of defeating LGR and MLP models with a small perturbation budget of 0.18. Additionally, the transfer attacks originating from the LGR model consistently outperformed those from the MLP model when targeting the other popular ML models. The PGD-assisted transfer attack based on the LGR model demonstrated similar attack performance to standard transfer attacks, but required significantly fewer iterations to generate adversarial examples. Furthermore, we compared the performance of the ZOO, Boundary, and HopSkipJump attacks. Our results show that the Boundary and HopSkipJump attacks significantly outperformed the ZOO attack, achieving a very high Attack Success Rate (ASR) across most of the ML models studied.

The main contributions and discoveries of this work are summarized as follows:

- We applied both white-box and black-box adversarial attacks originating from the image domain against nine ML-based NIDS models, including PGD, transfer attacks, PGD-assisted transfer attacks, ZOO, Boundary, and HopSkipJump attacks.
- The black-box decision-based attacks, such as Boundary and HopSkipJump attacks, proved to be highly effective in compromising the majority of the ML models studied in this work. Specifically, these attacks achieved an impressive success rate of over 86% across eight different ML models, highlighting their ability to exploit vulnerabilities even in scenarios where the attacker has no direct access to the model's structure or parameters.
- The instance-based ML models, such as KNN and LS, demonstrated a notable level of robustness against adversarial attacks. Unlike many other models, their resistance to adversarial perturbations stems from their reliance on local data structures and instance relationships, which makes it more challenging for adversarial manipulations to mislead their predictions. This characteristic highlights their potential as viable options in applications where security against adversarial attacks is critical.

The remainder of this paper is structured as follows: Section 2 reviews related work. Section 3 provides a detailed description of various adversarial attacks against NIDSs, along with the performance metrics used. Section 4 presents a performance evaluation of adversarial attacks on nine popular ML-based NIDS models using the NSL-KDD dataset. Finally, Section 5 concludes the paper and outlines directions for future work.

2. Related Work

This section reviews recent research on datasets used for ML-based NIDS models and popular adversarial attacks commonly applied in the image domain.

2.1. Datasets

The NIDS domain faces a shortage of available data, due to privacy and security concerns [2], limiting access to realistic and evolving training datasets. The most widely used datasets include the following:

(1) NSL-KDD [20]: This dataset is an improvement over the original KDD'99 dataset [21,22]. NSL-KDD contains both benign packets representing normal network traffic and malicious packets categorized into four types: Denial of Service (DoS), user-to-root, remote-to-local, and probing attacks. It features 41 attributes, including three categorical and 38 numerical features extracted from TCP/IP connections, traffic windows, and packet content.

(2) Kitsune [23]: This dataset includes nine network attack scenarios captured from an IP-based commercial surveillance system and IoT networks. The attacks include the

Mirai Botnet attack, comprising 764,137 packets targeting IoT devices, and the Video Injection MITM attack, with 2,473,401 packets. Kitsune provides both raw PCAP files and preprocessed CSV files, ready for ML analysis.

(3) CICIDS-2017 [24]: This dataset features a combination of benign traffic and common attacks, closely resembling real-world scenarios. It contains network traffic data labeled by timestamp, IP addresses, ports, and protocols, collected from 3 July to 7 July 2017. The attacks include Brute Force FTP/SSH, DoS, Heartbleed, Web Attack, Infiltration, Botnet, and DDoS, executed over five days.

The NSL-KDD dataset has been extensively used in research on Network Intrusion Detection Systems [25–28]. Moreover, many of the attack types represented in this dataset, such as Denial of Service and probing attacks, are also prevalent in IoT environments. Therefore, we utilized the NSL-KDD dataset in this study.

2.2. Adversarial Attacks

Adversarial attacks can be categorized into two types, based on the attacker's level of knowledge about the target ML model:

(1) White-box attacks: The attacker has full knowledge of the ML model, including its architecture and weights.

(2) Black-box attacks: The attacker does not know the ML model or its weights but can query the model to obtain output predictions.

White-box attacks were initially studied in the image domain. The following three white-box adversarial attacks have been widely applied:

(1) Fast Gradient Sign Method (FGSM) attack [12]: This attack adds a single perturbation to the input data, based on the sign of the derivative of the loss function with respect to the input, while adhering to a predefined limit on the magnitude of changes.

(2) PGD attack [15]: Also known as the basic iterative method, this attack is an iterative version of FGSM, applying multiple iterations to generate more effective adversarial examples.

(3) CW attack [19]: Unlike FGSM, the CW attack includes an image distance loss in the objective function (i.e., a targeted loss function). It also uses multiple iterations to produce the adversarial examples with minimum changes to the original image.

Black-box attacks are more challenging to execute than white-box attacks. Some notable black-box attacks include the following:

(1) Transfer attack [29]: In the image domain, it is well known that adversarial examples generated to attack one ML model can be transferred to attack another. This occurs because different models trained for the same task often learn similar non-robust features, which can be exploited by the same adversarial example. Transferability is particularly useful in practical scenarios, as attackers do not need direct access to the target model. In the context of NIDSs, Qiu et al. utilized model-extraction techniques to replicate the black-box model and generate adversarial examples targeting the original NIDS [11].

(2) ZOO attack [16]: A score-based attack, ZOO uses zeroth-order stochastic coordinate descent to identify adversarial examples.

(3) Boundary attack [17]: A decision-based attack, this method begins with a large perturbation and gradually reduces it to find an adversarial example with minimal perturbation.

(4) HopSkipJump attack [18]: An improved version of the Boundary attack, this method estimates gradients using binary search near the decision boundary, offering a novel approach to generating adversarial examples.

3. Adversarial Attacks Against NIDSs

In this study, we treated the detection of benign and malicious packets for NIDSs as a binary classification problem, where input examples are classified as either benign or

malicious. Creating adversarial examples involves altering the classification from malicious to benign or vice versa. Let the set of input examples to be perturbed be denoted as x. We define a classifier C that takes x as input and classifies it as either benign (0) or malicious (1), with the resulting prediction denoted as y. The objective is to find a perturbation δ such that, when introduced to the input example, it produces an adversarial example x', where the new prediction y' differs from y:

$$y = C(x) \tag{1}$$

$$x' = x + \delta \tag{2}$$

$$y' = C(x'), \quad \text{where} \quad y' \neq y \tag{3}$$

$$x, x' \in [0,1]^m \tag{4}$$

where m represents the number of input feature dimensions. Typically, input features for a classifier are scaled between 0 and 1.

3.1. White-Box Attacks

White-box attacks are used to generate adversarial examples with maximum attack effectiveness under a given perturbation budget, assuming that the attacker has full knowledge of the NIDS model and its weights. A model that can resist white-box adversarial attacks is considered highly robust, as finding adversarial examples is easier when the model is fully exposed. Some commonly used white-box attacks include FGSM [12], PGD [15], and CW [19] attacks. In this study, we focused on the PGD attack.

PGD Attacks

The PGD attack generates adversarial examples by exploiting the gradient of the model's loss function with respect to the input features. Specifically, it is an iterative attack that constrains the perturbation within an l_p ball after each iteration, while also clipping the input data if they exceed predefined bounds. This method was proposed by Madry et al. [15] as a way to train neural networks through adversarial training to enhance their robustness against attacks.

Let θ represent the set of parameters of the classifier C. The attack performs gradient descent on the loss function with respect to the input, modifying the input example at each iteration in the direction that alters the output prediction of the target NIDS model. Let x_i be the perturbed input example after the ith iteration. The input for the next iteration, x_{i+1}, is obtained as follows:

$$x_{i+1} = \Pi(x_i + \xi \, \text{sign}(\nabla_{x_i} L(\theta, x_i, y))) \tag{5}$$

Here, Π is the projection operator that ensures the perturbed input remains within the permissible bounds; $\nabla_{x_i} L(\theta, x_i, y)$ is the gradient of the loss function $L(\theta, x_i, y)$ with respect to the input x_i. The parameter ξ represents the step size for the gradient descent, and the sign function returns the sign of the input argument. After a sufficient number of iterations, the input example x may become an adversarial example x'. The attack seeks the maximum perturbation within a limit defined by ε, which sets the radius of the l_p-ball, i.e.,

$$||x' - x||_p \leq \varepsilon. \tag{6}$$

In this work, we refer to this limit, denoted as ε, as the perturbation budget. A smaller value of ε results in a more imperceptible adversarial attack.

In the context of cybersecurity, the features extracted from network packets can be perturbed to generate adversarial packets using the PGD attack. The restriction imposed by ε ensures that the perturbations remain realistic, preventing excessive modification of the packets and preserving the malicious characteristics of the adversarial examples. Moreover, the PGD attack cannot be directly applied to models with non-differentiable loss functions,

as it requires explicit calculation of $\nabla_{x_i} L(\theta, x_i, y)$. Therefore, in this study, we applied the PGD attack only to the LGR and MLP models.

In our experiments with the PGD attack, we observed that after several iterations the gradient $\nabla_{x_i} L(\theta, x_i, y)$ can become very small or even zero. As a result, the attack may cease making progress beyond a certain point, due to this vanishing gradient problem. Carlini and Wagner encountered a similar issue in their CW attacks [19]. To address this, we propose adding random Gaussian noise to the gradient when it approaches zero.

3.2. Black-Box Attacks

A black-box attack is a more practical approach, as in real-world scenarios, attackers typically do not have unrestricted access to the target NIDS. Without knowledge of the model, gradient information cannot be used to reliably generate adversarial examples. To overcome this limitation, researchers have developed several strategies, including transfer attacks, score-based attacks, and decision-based attacks.

3.2.1. Transfer Attacks

It has been shown that in the image domain, adversarial examples generated for one specific classifier can also be effective against another classifier, even if it has a different neural network architecture [12]. Building on this success, the process for generating adversarial examples against ML-based NIDSs can be outlined as follows:

(1) Define the task of the target model. In this case, it is a binary classification problem, where incoming packets are classified as either benign or malicious.
(2) Select an appropriate model that approximates the performance of the target model and has a differentiable loss function. Train this alternate classifier on the defined task.
(3) Apply the PGD attack, as discussed earlier, to the alternate classifier to generate adversarial examples. Many of these examples will successfully “transfer" to the target model and evade detection.

It is important to note that the transfer attack does not involve querying the target model during the generation of adversarial examples.

3.2.2. PGD-Assisted Transfer Attacks

The PGD-assisted transfer attack is similar to a standard transfer attack, in which a substitute ML model is used to approximate the target model. The key difference is that during the generation of adversarial examples from the substitute model, each example is tested against the target model to determine if it can evade detection. Only examples that successfully bypass the target model’s detection are retained. The detailed steps of this attack are outlined in Algorithm 1:

In Algorithm 1, Lines 6–9 highlight the key differences between the standard transfer attack and the PGD-assisted transfer attack. Lines 13–15 present our proposed enhancement to the PGD attack, where random Gaussian noise is added to the gradient when it becomes zero. Additionally, when the substitute classifier is identical to the target classifier, the PGD-assisted transfer attack simplifies to a standard PGD attack, which was discussed in Section PGD Attacks of Section 3.1.

Intuitively, adversarial examples generated by the PGD-assisted transfer attack are more effective against the target model than those produced by a standard transfer attack. However, the PGD-assisted transfer attack requires querying the target model once for each example at every iteration, leading to a significantly higher number of queries to the target ML model.

3.2.3. Score-Based Attacks

Score-based attacks do not have direct access to the internal structure of the target model, but they can query the model to obtain probability scores for the output classes. These attacks use the output scores to estimate the gradients of the loss function with respect to the input by querying data points near the original input. The ZOO attack is

Algorithm 1 PGD-Assisted Transfer Attack

Input: an original input x, output label y, target classifier C, substitute classifier M, loss function L, maximum iterations k, step size ξ, perturbation threshold ε
Output: an adversarial example $best_x'$
$best_x' \leftarrow x$
$x' \leftarrow x$
for $i = 0...k$ **do**
  $p' \leftarrow C(x')$
  if $p' \neq y$ **then**
    $best_x' \leftarrow x'$
  end if
  $p \leftarrow M(x')$
  $loss \leftarrow L(p, y)$
  $grad \leftarrow \xi \, \text{sign}(\nabla_{x'} loss)$
  if $grad = 0$ **then**
    $grad \leftarrow grad +$ Gaussian Noise N
  end if
  $x' \leftarrow x' + grad$
  $x' \leftarrow clip(x', x + \varepsilon, x - \varepsilon)$
  $x' \leftarrow clip(x', 0, 1)$
end for
return $best_x'$

a well-known example of a score-based attack that incorporates the principles of the CW attack using the L_∞ norm [16]. Unlike the CW attack, which depends on auto-differentiation through an optimizer, the ZOO attack calculates gradients numerically by applying finite differences to the model's outputs.

A random dimension i is selected, and a step is taken along that dimension. The gradient is estimated using the symmetric difference quotient g_i:

$$g_i = \frac{\partial f(x)}{\partial x_i} \approx \frac{f(x + le_i) - f(x - le_i)}{2l} \tag{7}$$

Here, l is a small constant, and e_i is the basis vector with the ith dimension set to 1 and all other dimensions set to 0.

Consider an input example with a total of m dimensions. A straightforward approach to estimating the gradient is to apply the above equation to each dimension i. However, this method is computationally expensive because it requires computing f twice for each dimension, resulting in a total of $2m$ calls. To address this inefficiency, the attack also utilizes a coordinate-wise Hessian estimate and applies Newton's method to approximate the gradient more efficiently. The Hessian estimate is calculated as follows:

$$h_i = \frac{\partial^2 f(x)}{\partial x_i^2} \approx \frac{f(x + le_i) - 2f(x) + f(x - le_i)}{l^2} \tag{8}$$

When h_i is greater than 0, f is convex, and we can update the dimension in the appropriate direction, using Newton's method:

$$\delta = -\xi \frac{g_i}{h_i} \tag{9}$$

where ξ is the step size. However, if h_i is less than or equal to zero then f is concave and a larger step can be taken by moving along the direction of the gradient:

$$\delta = -\xi g_i \tag{10}$$

This approach reduces the number of calls to two per update step. The ZOO attack is detailed in Algorithm 2:

Algorithm 2 ZOO Attack [16]

Input: an original input x, step size ξ
Output: an adversarial example x'
$x' \leftarrow x$
while not converged **do**
  Pick random coordinate i $\in$ m
  Estimate g_i and h_i from Equations (7) and (8)
  if $h_i \leq 0$ **then**
    $\delta \leftarrow -\xi \times g_i$
  else
    $\delta \leftarrow -\xi \times \frac{g_i}{h_i}$
  end if
  $x'_i \leftarrow x'_i + \delta$
end while
return x'

3.2.4. Decision-Based Attacks

Decision-based attacks operate under stricter constraints, where only the ML model's output labels are accessible, without access to probability scores. Assuming unlimited access to a model's output labels, one approach to attacking the NIDS model involves adding perturbations to input examples and searching the surrounding space by repeatedly querying the target model to see if the output label changes. However, this random search method is highly inefficient.

To significantly reduce the number of queries, decision-based attacks employ a more refined approach. They start by finding an example from the target class, denoted as x''. The original example x is then perturbed along the line connecting x to x''. At some point, the perturbed example reaches the boundary between two classes. From there, the example is projected along the boundary to identify adversarial examples.

Boundary Attacks

The Boundary attack [17] is an example of a decision-based attack. It selects a perturbation η drawn from a proposal distribution P. The choice of this proposal distribution is crucial to the algorithm's efficiency, as it guides the generation of viable perturbations and directs the search around the decision boundary.

In their work, the authors of [17] selected a simple maximum-entropy distribution, subject to the following three constraints S, as their proposal distribution:

(1) The perturbed example remains within the domain of the input:

$$x' + \eta \in (0,1)^m \tag{11}$$

where m is the number of dimensions.

(2) The perturbation has a relative magnitude of δ:

$$||\eta|| = \delta \times d(x, x') \tag{12}$$

where $d(x, x')$ is the distance between x and x', which can be computed using any L_p norm.

(3) The perturbation shifts the example by a relative amount of r:

$$d(x, x') - d(x, x' + \eta) = r \times d(x, x'). \tag{13}$$

The attack starts with an initial point x', randomly selected from the target class, which is typically distant from the original example x. If the perturbation η is adversarial, it is projected onto a sphere around the original example x through orthogonal projection. The point x' is then gradually brought closer to the original example in an iterative process. This procedure is repeated for k steps, yielding a robust x' that remains adversarial while satisfying the constraints of the L_2 or L_∞ distance metrics. The implementation of the Boundary attack is detailed in Algorithm 3.

Algorithm 3 Boundary Attack [17]

Input: an original input x, classifier C, set of constraints S, maximum iterations k
Output: an adversarial example x^k
$x^0 \leftarrow$ drawn from distribution: $x^0 \in$ target class
for $i = 1...k$ **do**
  $\eta^i \leftarrow$ drawn from proposal distribution $P(x^{i-1})$ following S
  if $C(x^{i-1} + \eta^i) \in$ target class **then**
    $x^i \leftarrow x^{i-1} + \eta^i$
  else
    $x^i \leftarrow x^{i-1}$
  end if
end for
return x^k

HopSkipJump Attacks

The HopSkipJump attack [18] is an advanced variant of the Boundary attack that enhances performance by incorporating a novel binary search approach at the decision boundary. The initial phase of the HopSkipJump attack resembles the Boundary attack, starting with a sample drawn from a random distribution. For an untargeted attack, the classifier outputs a misclassification of the current class, whereas for a targeted attack the sample is chosen from the target class. The attack then iteratively performs the following steps:

(1) Binary Search: In the i-th iteration, the point x_{i-1} from the previous iteration is moved closer to the boundary through a binary search.
(2) Gradient Approximation: The gradient direction at the boundary is estimated using a novel technique that calculates a weighted average of vectors sampled from an independently and identically distributed (i.i.d.) draw over the m-dimensional sphere at the boundary point. This estimate is unbiased only at the decision boundary, which is why the boundary point is first identified via binary search.
(3) Step Size Initialization: A step size is initialized and iteratively updated using a geometric progression until the perturbation is successful.

The resulting adversarial sample is projected back onto the boundary.

Notably, this attack does not require any score estimates, as it approximates the gradients by analyzing label outputs with a series of small perturbations at the boundary. The implementation details of the HopSkipJump attack are outlined in Algorithm 4:

3.3. Performance Metrics

We defined several performance metrics, to assess the strength and quality of the adversarial examples. These metrics evaluated the effectiveness of the adversarial examples against the target NIDS models, as well as the extent of the changes made from the original examples.

We first assessed the attack strength of the adversarial examples on the target model by calculating the ratio of the examples that successfully fooled the model to the total number of generated examples. This ratio, expressed as a percentage, is known as the Attack Success Rate (ASR). A higher ASR indicated a stronger adversarial attack, with more adversarial examples able to bypass the model.

Algorithm 4 HopSkipJump Attack [18]

Input: an original input x, classifier C, constraint L_p, iterations k
Output: an adversarial example x'
$x_0 \leftarrow$ initialized as $x_0 \in$ target class
$d_0 \leftarrow ||x_0 - x||_p$
for $i = 1...k$ **do**
  Push x_{i-1} to boundary, i.e.,
  $x'_i \leftarrow$ BinarySearch(x_{i-1}, x)
  Sample $B_i \leftarrow u_1, u_2, ...$
  $\nu_i \leftarrow$ Estimate gradient using B_i and C (step-size search)
  $\xi_i \leftarrow d_{i-1} \ / \ \sqrt{i}$
  $x_i \leftarrow x'_i + \xi_i \nu_i$
  while x_i is not adversarial **do**
    $\xi_i \leftarrow \xi_i / 2$
    $x_i \leftarrow x'_i + \xi_i \nu_i$
  end while
  $d_i \leftarrow ||x_i - x||_p$
end for
return $x' \leftarrow$ BinarySearch(x_k, x)

Specifically, we let x be a set of input examples, with $x_0 \subset x$ representing the subset selected for evaluation. We denoted the target model as the classifier C, which the adversarial attack aimed to deceive. Note that x_0 was the dataset in which all examples were correctly classified by the target model. The attack was applied to x_0, to generate a perturbed set x'_0. Within x'_0, we let x_a represent the adversarial examples that successfully attacked the model. For a targeted attack, x_a was classified as the target class by the NIDS model, while for an untargeted attack, x_a was classified as a different class from the original. The ASR for a targeted attack was defined as follows:

$$\text{ASR} = \frac{Count(x_a)}{Count(x_0)} \tag{14}$$

$$C(x_a) \in \text{target class}, \quad x_a \subseteq x'_0 \tag{15}$$

For an untargeted attack, the ASR was defined as

$$\text{ASR} = \frac{Count(x_a)}{Count(x_0)} \tag{16}$$

$$C(x_a) \notin \text{original class}, \quad x_a \subseteq x'_0 \tag{17}$$

We also evaluated the strength of the adversarial examples using the accuracy metric, which measured the ratio of the examples that the target NIDS model could correctly classify out of the total number of examples. When considering only x'_0, the accuracy was equal to $1 - \text{ASR}$. However, if the input dataset included examples that were misclassified by the model before the attack, the accuracy did not equal $1 - \text{ASR}$. For instance, if a classifier achieved 80% accuracy on a dataset prior to an adversarial attack, and if the attack did not alter the outputs of the NIDS model for any of the input examples, then the ASR would be 0 but the accuracy would remain 80%. Therefore, to accurately assess the ASR, we had to focus exclusively on the subset of input examples that were correctly classified before the attack was applied.

Additionally, we evaluated the imperceptibility of perturbations in the adversarial examples. In the network domain, if a malicious packet was perturbed, to be classified as benign by the NIDS the perturbation should not be so large that the packet lost its malicious properties and effectively became benign. Therefore, an adversarial example was considered strong if it introduced an imperceptible perturbation while still being

misclassified by the model. Such an example retained the intrinsic characteristics of its original class despite being misclassified.

To quantify the differences between the original and perturbed examples, we defined distance metrics using the L_p norms, specified as follows:

$$||z||_p = \left(\sum_{i=1}^{n} |z_i|^p \right)^{\frac{1}{p}} \tag{18}$$

where the L_2 norm represented the Euclidean distance, and the L_∞ norm was defined as

$$||z||_\infty = \max_i |z_i| \tag{19}$$

The L_p norms were effective for measuring the distance between x and x', and they are commonly used in popular adversarial attacks to assess the imperceptibility of perturbations. A smaller L_p distance indicated a less perceptible adversarial example.

4. Experimental Results

In this section, we first outline the preparation of the NSL-KDD dataset and the development of the ML-based NIDS models. Next, we present our evaluation of the performance of PGD white-box adversarial attacks against the NIDS. Finally, we present our assessment of the effectiveness of the various black-box attacks.

4.1. Dataset Preparation and ML-Based NIDS

In this study, we focused on the binary classification task of identifying input packets as either benign or malicious, using the NSL-KDD dataset [20]. The NSL-KDD dataset is an improved version of the KDD Cup 1999 dataset [21,22], with redundancies and duplicates removed to reduce bias. It contains 125,973 inputs in the training dataset and 22,544 entries in the test dataset. We analyzed the distribution of the benign and malicious packets, as illustrated in Figure 1. Notably, the test dataset in NSL-KDD had a higher percentage of malicious packets compared to the training dataset. Additionally, we observed that the test dataset included 14 types of attacks not present in the training dataset, which originally contained 24 attack types. This added realism to the detection task, as NIDS models must be capable of identifying and defending against previously unseen attacks.

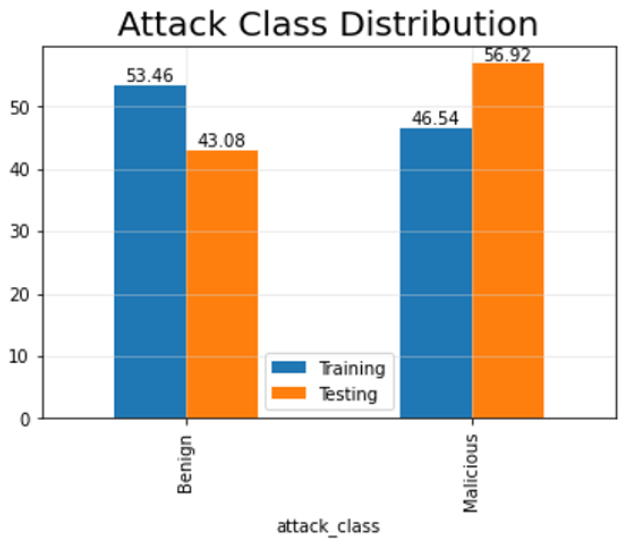


Figure 1. Proportion of benign and malicious packets in NSL-KDD training and test datasets.

To prepare the data, we performed the following data preprocessing steps:

- Mapping the various output labels to binary values, where benign was represented as 0 and malicious as 1.
- Converting three categorical features (i.e., protocol type, flag, and service) into numerical values, using one-hot encoding, resulting in a total of 121 numerical features for each input.
- Scaling all input features to a range from 0 to 1, using a min–max scaler.

After preparing the NSL-KDD dataset, we trained nine popular ML classifiers, using the training set, and we evaluated their performance on the test set. The ML models studied included LGR, MLP, SVM with a linear kernel, SVM with an RBF kernel, DTC, RFC, XGBoost, KNN, and LS. The XGBoost model was implemented using the XGBoost library [30], while the remaining models were adopted from the scikit-learn library [31]. For each model, where applicable, we set the random state to 42 prior to training. Default hyper-parameter values from the libraries were used for the ML models, except for the following adjustments:

- LGR: Maximum number of iterations set to 2000.
- MLP: Configured with a single hidden layer containing 150 nodes and a maximum number of iterations set to 1000.
- DTC: Criterion set to "entropy".
- XGBoost: Objective set to "binary:logistic".
- LS: Gamma set to 3.

We evaluated the performance of these ML models on both the training and test datasets by measuring accuracy (expressed as "Acc") in percentage, precision, recall, Area Under the receiver operating characteristic Curve (AUC), and F1 score [14], as shown in Table 1.

From Table 1, we observe a noticeable drop in accuracy and F1 scores on the test dataset compared to the training dataset, reflecting differences in the data distributions. Additionally, the F1 scores for the test dataset ranged from 0.74 to 0.81 across all nine models, indicating that the ML models exhibited similar detection performance on the NSL-KDD dataset. Among them, the DTC model, with its simple and rule-based structure, achieved the highest generalization performance, with an F1 score of 0.81 and an AUC of 0.83. These classifiers will serve as the target NIDS models for subsequent experiments on generating and evaluating adversarial examples.

Table 1. Performance evaluation of nine ML-based NIDS models on training and test datasets ("Acc" refers to accuracy, while "AUC" stands for the Area Under the Curve) .

Model	Training Dataset					Test Dataset				
	Acc	**Precision**	**Recall**	**AUC**	**F1**	**Acc**	**Precision**	**Recall**	**AUC**	**F1**
LGR	97.3	0.980	0.962	0.972	0.971	75.2	0.917	0.621	0.773	0.740
MLP	99.8	0.998	0.997	0.997	0.997	81.1	0.965	0.693	0.789	0.762
SVM-linear	97.4	0.984	0.960	0.973	0.972	75.2	0.916	0.621	0.773	0.740
SVM-rbf	98.6	0.989	0.981	0.986	0.985	76.4	0.920	0.640	0.783	0.755
DTC	99.9	0.999	0.999	0.999	0.999	81.3	0.971	0.693	0.833	0.808
RFC	99.9	0.999	0.999	0.999	0.999	78.0	0.969	0.635	0.804	0.767
XGBoost	99.9	0.999	0.999	0.999	0.999	78.0	0.968	0.634	0.789	0.762
KNN	99.7	0.998	0.997	0.997	0.997	76.9	0.924	0.648	0.788	0.761
LS	98.2	0.989	0.972	0.981	0.980	76.9	0.922	0.650	0.789	0.762

4.2. PGD Attacks

We trained a new LGR model and a new MLP model. The MLP model featured two hidden layers with 128 and 64 nodes, respectively, both using the ReLU activation function. Additionally, it employed binary cross-entropy loss, the Adam optimizer, and a sigmoid activation function for the output layer. These two models were used to generate and evaluate adversarial examples. To differentiate these models from the original NIDS models, we refer to them as LGR_s and MLP_s. We evaluated the performance of the PGD attacks by attacking both models using 1000 samples from the test dataset. In a PGD attack, the binary cross-entropy loss function was used, with a maximum of 5000 iterations and a step size of $\varepsilon/50$, where ε was the perturbation budget. Our study examined how increasing the perturbation budget ε affected the model prediction accuracy. We also analyzed the L_2 and L_∞ distances between the original input and the adversarial example as ε increased.

Figures 2 and 3 present the experimental results for the LGR_s model. As shown in Figure 2, an increase in ε resulted in the generation of more successful adversarial examples by the PGD attack, leading to a decline in accuracy. Specifically, when ε reached 0.15 the PGD attack achieved 0% accuracy, corresponding to a 100% ASR. Furthermore, Figure 3 demonstrates that the L_2 and L_∞ distances increased almost linearly with ε.

Figures 4 and 5 present the experimental results for the MLP_s model. Similar to the LGR_s model, as ε increased the success rate of the PGD attack against the MLP_s model rose, while the L_2 and L_∞ distances grew almost linearly. Specifically, when ε reached 0.18, the model's accuracy dropped to 0%, corresponding to a 100% ASR. However, unlike the LGR_s model, the accuracy for the MLP_s model remained stable or changed only slightly when ε was between 0.03 and 0.1. This behavior may be attributed to the complex properties of neural networks [12].

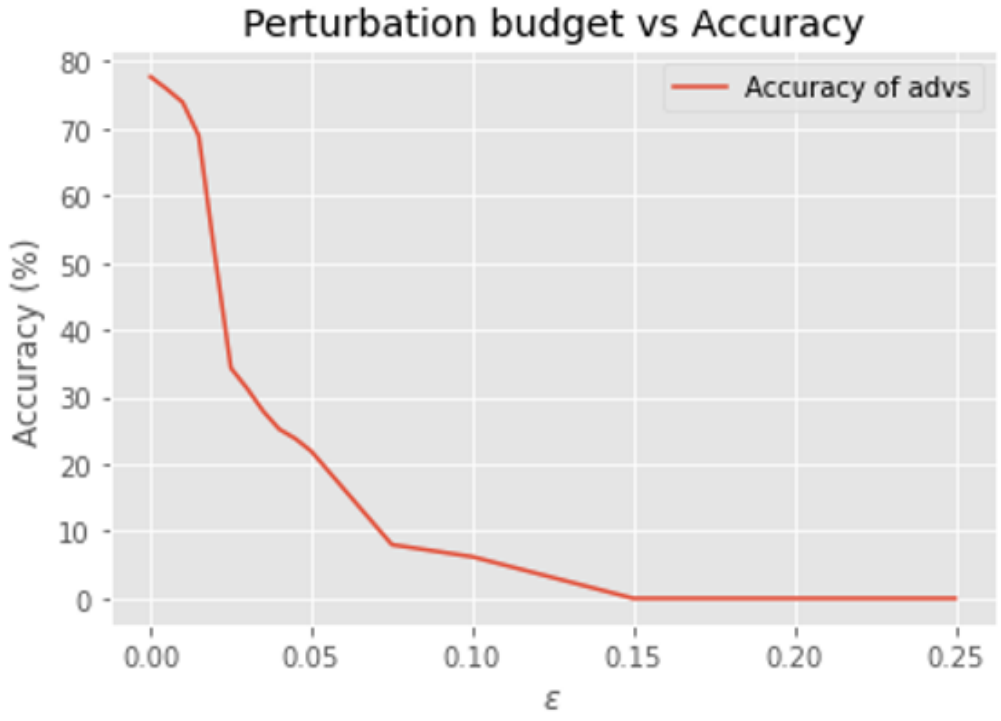


Figure 2. The detection accuracy of the LGR_s model under adversarial attacks with different perturbation budgets ε.

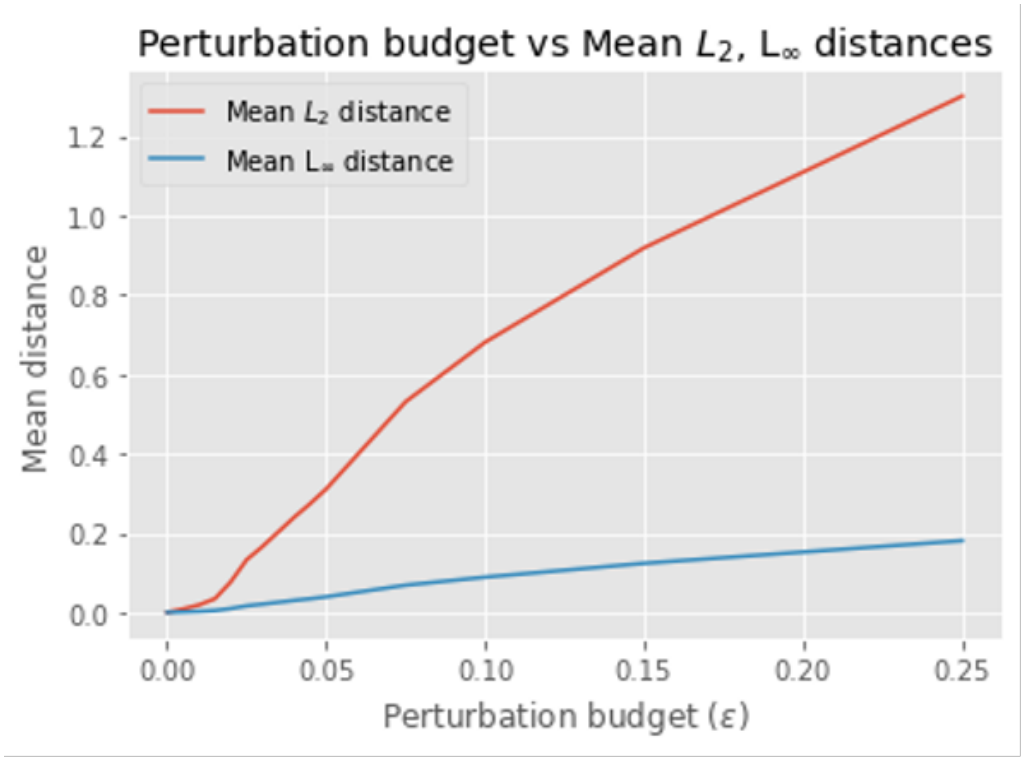


Figure 3. The L_2 and L_∞ of adversarial examples with different perturbation budgets ε against the LGR_s model.

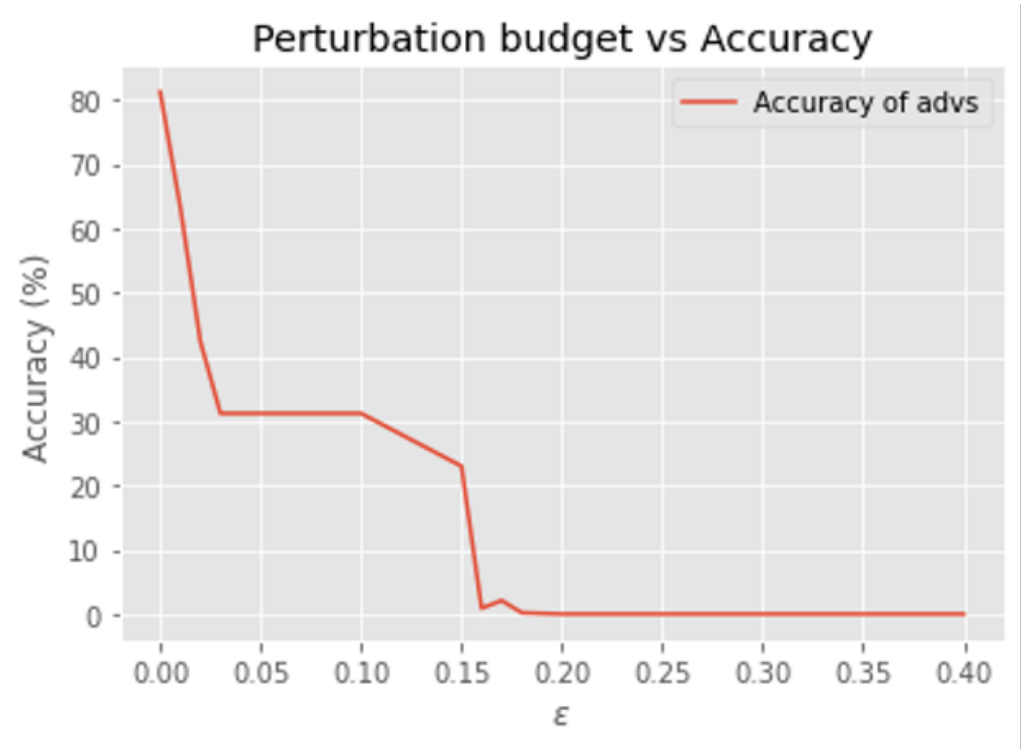


Figure 4. The detection accuracy of the MLP_s model under adversarial attacks with different perturbation budgets ε.

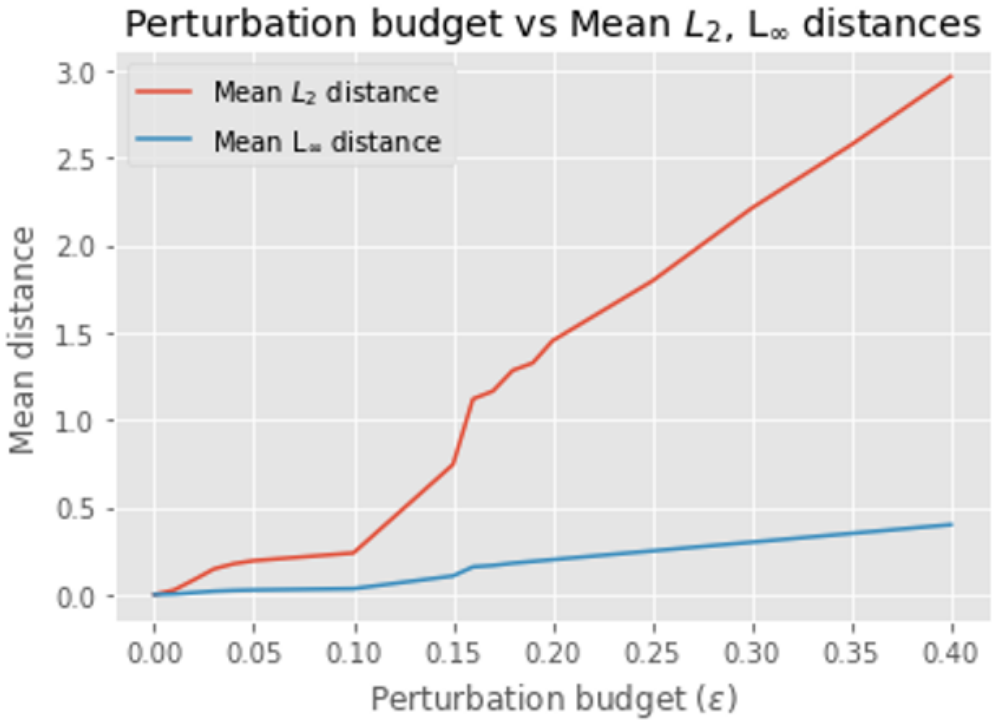


Figure 5. The L_2 and L_∞ of adversarial examples with different perturbation budgets ε against the MLP_s model.

Figures 2 and 4 show that the PGD white-box attack could achieve a 100% ASR against both the LGR_s and the MLP_s models with a relatively small perturbation budget, such as $\varepsilon = 0.18$.

4.3. Black-Box Attacks

4.3.1. Transfer Attacks

We examined the transferability of the adversarial examples generated by the PGD attacks on the LGR_s and MLP_s models to the other nine ML-based NIDS models. As shown in the previous section, all the examples generated by the PGD attack were fully adversarial for both $\varepsilon = 0.3$ and $\varepsilon = 0.4$ against the LGR_s and MLP_s models. Table 2 presents the performance of those adversarial examples on the nine ML-based NIDS models. We observe that the adversarial examples generated using the LGR_s model outperformed those generated by the MLP_s model for all the ML models except RFC. Additionally, the perturbation budget, whether $\varepsilon = 0.3$ or 0.4, had little impact on most of the results. It is noted that as ε increased from 0.3 to 0.4, the ASR of the transfer attacks from the LGR_s model against the DTC model decreased from 81.8% to 46.9%. This decline may be attributed to "overfitting" of the LGR_s model on the generated adversarial examples, reducing their effectiveness against the DTC model.

Notably, the adversarial examples crafted for the LGR_s model with $\varepsilon = 0.4$ achieved a 100% ASR against the LGR, MLP, SVM-linear, and SVM-rbf models, and an 84% ASR against the XGBoost model. Additionally, the RFC and KNN models displayed relative robustness against these transfer attacks, while the LS model showed strong resistance.

Table 2. Performance evaluation of transfer attacks with different perturbation budgets against ML-based NIDS models.

Model	**Original Acc**	**$LGR_s(\varepsilon = 0.3)$**		**$LGR_s(\varepsilon = 0.4)$**		**$MLP_s(\varepsilon = 0.3)$**		$MLP_s(\varepsilon = 0.4)$	
Target NIDS		**Acc**	**ASR**	**Acc**	**ASR**	**Acc**	**ASR**	**Acc**	**ASR**
LGR	77.3	0.8	98.9	0.0	100.0	24.0	69.0	18.4	76.2
MLP	83.9	0.9	98.9	0.0	100.0	30.0	64.2	27.3	67.5
SVM-linear	77.1	0.5	99.4	0.0	100.0	18.4	76.1	17.7	77.0
SVM-rbf	78.3	12.9	83.5	0.0	100.0	28.8	63.2	21.3	72.8
DTC	84.5	16.4	81.8	45.6	46.9	49.0	47.6	48.6	49.9
RFC	81.3	63.0	22.5	53.2	34.6	39.3	52.5	40.7	50.7
XGBoost	81.9	18.6	80.2	15.1	84.0	33.6	62.3	31.1	64.6
KNN	79.6	50.9	36.3	44.8	43.9	50.6	37.4	50.2	39.9
LS	79.1	72.1	8.8	66.4	16.1	77.0	2.7	75.3	4.8

4.3.2. PGD-Assisted Transfer Attacks

We applied the PGD-assisted transfer attacks, as defined in Algorithm 1, against the nine ML-based NIDS models, using the adversarial examples generated by the LGR_s or MLP_s models with $\varepsilon = 0.3$ and $\varepsilon = 0.4$. Table 3 presents the evaluation results of the PGD-assisted transfer attacks.

For the adversarial examples generated by the LGR_s model, the accuracy and ASR results in Table 3 are similar to those in Table 2. However, overall, the PGD-assisted transfer attacks performed less efficiently than the standard transfer attacks for the MLP_s model. This was because each PGD-assisted transfer attack used significantly fewer iterations. Specifically, a transfer attack ran for 10,000 iterations to find adversarial examples, while a PGD-assisted transfer attack ran for only 2000 iterations.

Table 3. Performance evaluation of PGD-assisted transfer attacks with different perturbation budgets against ML-based NIDS models.

Target NIDS	**$LGR_s(\varepsilon = 0.3)$**				**$LGR_s(\varepsilon = 0.4)$**				**MLP_s $(\varepsilon = 0.3)$**				**$MLP_s(\varepsilon = 0.4)$**			
	Acc	**ASR**	**L_2**	**L_∞**	**Acc**	**ASR**	L_2	**L_∞**	**Acc**	**ASR**	L_2	**L_∞**	**Acc**	**ASR**	L_2	**L_∞**
LGR	0.8	99.0	2.3	0.3	0.0	100.0	3.0	0.4	72.6	6.1	0.8	0.1	67.4	12.8	1.1	0.2
MLP	0.6	99.3	2.3	0.3	0.0	100.0	3.0	0.4	47.1	44.5	0.8	0.1	47.6	45.4	1.1	0.2
SVM-linear	0.0	100.0	2.3	0.3	0.0	100.0	3.0	0.4	68.0	11.8	0.8	0.1	62.5	18.9	1.1	0.2
SVM-rbf	11.3	85.6	2.3	0.3	0.0	100.0	3.0	0.4	70.5	9.9	0.8	0.1	68.1	13.0	1.1	0.2
DTC	16.4	81.8	2.3	0.3	45.6	46.9	3.0	0.4	52.1	46.8	0.8	0.1	53.6	46.3	1.1	0.2
RFC	63.2	22.3	2.3	0.3	53.2	34.6	3.0	0.4	45.2	46.2	0.8	0.1	44.8	46.2	1.1	0.2
XGBoost	17.4	81.6	2.3	0.3	15.1	84.0	3.0	0.4	34.4	62.6	0.8	0.1	33.7	62.1	1.1	0.2
KNN	50.9	36.3	2.3	0.3	44.8	44.0	3.0	0.4	70.9	11.1	0.8	0.1	71.0	11.1	1.1	0.2
LS	72.1	8.8	2.3	0.3	66.1	16.4	3.0	0.4	77.0	2.7	0.8	0.1	75.3	4.8	1.1	0.2

Similar to the standard transfer attacks, the PGD-assisted transfer attacks based on the LGR_s model were more successful at generating adversarial examples than those based on the MLP_s model, except against the RFC model. Additionally, the adversarial examples generated by the PGD-assisted transfer attacks using LGR_s with $\varepsilon = 0.4$ achieved a 100% ASR against the LGR, MLP, SVM-linear, and SVM-rbf models, and an 84% ASR against the XGBoost model. However, the LS model demonstrated strong robustness against the PGD-assisted transfer attacks, while the RFC and KNN models showed relative resistance.

It is important to note that the PGD-assisted transfer attacks were implemented using the L_∞ norm for the perturbation budget ε. Consequently, all the L_∞ values in Table 3 are less than or equal to ε. Additionally, the L_2 values for the LGR_s model were higher than those for the MLP_s model.

4.3.3. Score-Based and Decision-Based Attacks

We examined three score-based and decision-based black-box attacks—ZOO, Boundary, and HopSkipJump—against nine ML-based NIDS models. These attacks were implemented using the Adversarial Robustness Toolbox (ART) [32]. This toolbox offers two different configurations for black-box attacks: untargeted and targeted. In our experiments, when using the ART library for a ZOO attack, we set the confidence to 0.01, the maximum number of iterations to 50, and the binary search steps to 20, while disabling both the resizing and importance parameters. For a Boundary attack, we specified 50 as the maximum number of iterations and 128 for the batch size. For the HopSkipJump attack, all default parameter values were used.

Table 4 presents the performance evaluation results of the three attacks in the untargeted configuration. As shown, the Boundary and HopSkipJump attacks generally outperformed the ZOO attack, exhibiting lower accuracy and higher ASR for most of the ML models. Specifically, the ZOO attack failed to successfully target the SVM-linear, SVM-rbf, DTC, RFC, and KNN models. The Boundary and HopSkipJump attacks had similar performance, both achieving an ASR above 86% for all the ML models except the SVM-rbf model. Compared to the Boundary attack, the HopSkipJump attack required significantly fewer queries to the target model to estimate gradients but was much slower in terms of processing time.

Table 4. Performance evaluation of ZOO, Boundary, and HopSkipJump attacks against ML-based NIDS model using the untargeted black-box configuration.

Model	Original		ZOO Attack					Boundary Attack					HopSkipJump Attack				
	Acc	F1	Acc	ASR	F1	L_2	L_∞	Acc	ASR	F1	L_2	L_∞	Acc	ASR	F1	L_2	L_∞
LGR	77.3	0.760	22.7	100.0	0.328	0.3	0.1	24.8	97.2	0.360	0.4	0.1	24.2	97.9	0.351	0.3	0.1
MLP	83.9	0.833	30.5	82.5	0.456	0.1	0.0	27.1	86.2	0.418	0.3	0.1	26.0	87.8	0.401	0.2	0.1
SVM-linear	77.1	0.758	77.2	0.0	0.760	0.0	0.0	22.9	100.0	0.329	0.3	0.1	23.1	99.7	0.332	0.2	0.1
SVM-rbf	78.3	0.773	78.4	0.0	0.774	0.0	0.0	54.8	52.9	0.708	0.2	0.1	54.8	52.9	0.708	0.2	0.1
DTC	84.5	0.839	79.8	9.2	0.782	0.0	0.0	15.5	100.0	0.256	0.2	0.1	15.5	100.0	0.256	0.2	0.2
RFC	81.3	0.799	70.3	21.0	0.694	0.1	0.1	18.7	100.0	0.302	0.5	0.2	18.7	100.0	0.302	0.2	0.2
XGBoost	81.9	0.807	45.1	66.3	0.373	0.0	0.0	18.1	100.0	0.292	0.1	0.0	18.1	100.0	0.292	0.0	0.0
KNN	79.6	0.789	79.4	0.3	0.787	0.0	0.0	20.4	100.0	0.294	1.0	0.4	20.4	100.0	0.294	1.0	0.5
LS	79.1	0.783	44.1	70.5	0.576	1.4	0.6	20.9	100.0	0.302	1.6	0.6	20.9	100.0	0.302	1.5	0.6

Table 4 also indicates that compared to decision-based attacks the ZOO attack aimed to generate adversarial examples with smaller perturbations, as evidenced by the lower L_2 and L_∞ values. However, the reduced perturbation may have limited the effectiveness of the ZOO attack.

For the ML-based NIDS models, LGR and MLP were highly vulnerable to all three attacks. SVM-linear, DTC, and RFC were highly resistant to the ZOO attack but highly vulnerable to the decision-based attacks. Additionally, the SVM-rbf model was highly robust against the ZOO attack, with a 0% ASR, and it was relatively resistant to the decision-based attacks, with an ASR of 52.9%. XGBoost was moderately vulnerable to the ZOO attack but highly susceptible to the decision-based attacks. Furthermore, KNN was highly robust against the ZOO attack, while LS was moderately vulnerable to it. Both the KNN and LS models were highly vulnerable to the decision-based attacks but required relatively large perturbation budgets.

Table 5 presents the performance evaluation results of the three attacks in the targeted configuration. When comparing the results in Table 5 with those in Table 4, we observe that the attack performance in terms of ASR was similar for both the untargeted and the targeted configurations. However, providing target labels to the decision-based attacks significantly reduced model accuracy. Specifically, the accuracy of the ML-based NIDS models dropped to zero or nearly zero for both the Boundary and HopSkipJump attacks, with the exception of the MLP and SVM-rbf models. Additionally, the L_2 and L_∞ distances were smaller in the targeted configuration compared to the untargeted configuration for all three attacks, improving the imperceptibility of the adversarial examples.

Notably, apart from the KNN and LS models, the adversarial examples generated fell within an L_∞ distance of 0.2. While the decision-based attacks still successfully bypassed KNN and LS, they require relatively large perturbation budgets to do so. This suggests that instance-based ML models may exhibit unique robustness against adversarial attacks.

Table 5. Performance evaluation of ZOO, Boundary, and HopSkipJump attacks against ML-based NIDS model using the targeted black-box configuration.

Model	Original		ZOO Attack					Boundary Attack					HopSkipJump Attack				
	Acc	F1	Acc	ASR	F1	L_2	L_∞	Acc	ASR	F1	L_2	L_∞	Acc	ASR	F1	L_2	L_∞
LGR	77.3	0.760	0.0	100.0	0.000	0.2	0.1	1.7	97.8	0.033	0.3	0.1	1.5	98.1	0.030	0.2	0.1
MLP	83.9	0.833	14.2	83.1	0.246	0.1	0.0	10.1	87.9	0.183	0.3	0.1	9.6	88.6	0.175	0.2	0.1
SVM-linear	77.1	0.758	77.1	0.0	0.758	0.0	0.0	0.0	100.0	0.000	0.2	0.1	0.0	100.0	0.000	0.2	0.1
SVM-rbf	78.3	0.773	78.3	0.0	0.773	0.0	0.0	36.9	52.9	0.539	0.1	0.1	36.9	52.9	0.539	0.1	0.1
DTC	84.5	0.839	76.7	9.2	0.743	0.0	0.0	0.0	100.0	0.000	0.2	0.1	0.0	100.0	0.000	0.2	0.2
RFC	81.3	0.799	65.4	19.6	0.614	0.1	0.0	0.0	100.0	0.000	0.4	0.1	0.0	100.0	0.000	0.2	0.1
XGBoost	81.9	0.807	27.9	65.9	0.000	0.0	0.0	0.0	100.0	0.000	0.1	0.0	0.0	100.0	0.000	0.0	0.0
KNN	79.6	0.789	79.4	0.3	0.787	0.0	0.0	0.0	100.0	0.000	0.8	0.3	0.0	100.0	0.000	0.8	0.4
LS	79.1	0.783	23.8	69.9	0.355	1.1	0.5	0.0	100.0	0.000	1.4	0.5	0.0	100.0	0.000	1.2	0.5

4.4. Summary

We summarize our experimental results in Table 6. A model was considered highly vulnerable if the attack achieved a high ASR (75–100%). It was classified as moderately vulnerable if a significant portion of the adversarial examples were successful (25–75%). A model was deemed to have low vulnerability if only a small percentage (0–25%) of the adversarial examples were successful.

Table 6. Robustness of nine ML-based NIDS models against different adversarial attacks (H—High vulnerability, M—Moderate vulnerability, L—Low vulnerability, NA—Not Applicable).

ML Models	PGD Attacks	Transfer Attacks	ZOO Attack	Decision-Based Attacks
LGR	H	H	H	H
MLP	H	H	H	H
SVM-linear	NA	H	L	H
SVM-rbf	NA	H	L	M
DTC	NA	H	L	H
RFC	NA	M	L	H
XGBoost	NA	H	M	H
KNN	NA	M	L	H (large ε)
LS	NA	L	M	H (large ε)

Essentially, if an adversarial attack achieves a high ASR against an ML-based NIDS then an attacker could modify their malicious traffic to evade detection by the NIDS. Consequently, the ML-based NIDSs became highly vulnerable to such attacks.

5. Conclusions and Future Work

In this study, we systematically explored the generation and evaluation of adversarial examples in the context of NIDSs. We established a baseline by training nine popular ML models using the NSL-KDD dataset, and we evaluated their performance against potential adversarial examples. Specifically, we examined both white-box and black-box adversarial attacks on ML-based NIDS models. This study emphasized the challenges of translating adversarial attack techniques from the image domain to the NIDS domain. Moreover, we introduced a modification of the transfer attack—namely, the PGD-assisted transfer attack—to expedite the search for adversarial examples. Additionally, we compared various methods of generating adversarial examples by assessing model accuracy, ASR, and L_p distances. To the best of our knowledge, this is the first study to systematically evaluate adversarial attacks from the image domain against ML-based NIDSs, incorporating a broad

selection of ML models. A notable finding was that among the nine ML models studied, the instance-based models such as KNN and LS demonstrated greater overall robustness against adversarial attacks compared to the other models. Additionally, the Boundary and HopSkipJump attacks were highly effective against most of the ML models, achieving an attack success rate exceeding 86% across eight models.

As part of our ongoing work, we will incorporate the masking mechanism when processing the NSL-KDD dataset [26]. Specifically, during the generation of adversarial examples, certain input features will remain unchanged, based on practical considerations. Meanwhile, we plan to extend this study to include other network datasets, such as the Kitsune [23] and CICIDS-2018 [24] datasets. Furthermore, we will explore additional adversarial attack methods against ML-based NIDS models, such as the Jacobian-based saliency map attack [33]. We aim to identify the ML-based NIDS model that is most robust against a variety of network intrusion and adversarial attacks.

Author Contributions: Conceptualization, S.S. and Z.C.; methodology, S.S. and Z.C.; software, S.S. and Z.C.; validation, S.S. and Z.C.; formal analysis, S.S. and Z.C.; investigation, S.S. and Z.C.; resources, S.S. and Z.C.; data curation, S.S. and Z.C.; writing—original draft preparation, S.S. and Z.C.; writing—review and editing, S.S. and Z.C.; visualization, S.S.; supervision, Z.C.; project administration, Z.C.; funding acquisition, Z.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by a 2023 Purdue University Fort Wayne Graduate Research Assistantship.

Data Availability Statement: The original data presented in the study are openly available in https://www.kaggle.com/datasets/hassan06/nslkdd (accessed on 18 December 2024).

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

ASR	Attack Success Rate
AUC	Area Under the Curve
CW	Carlini and Wagner
DL	Deep Learning
DTC	Decision Tree Classifier
FGSM	Fast Gradient Sign Method
KNN	K-Nearest Neighbors
LGR	Logistic Regression
LS	Label Spreading
ML	Machine Learning
MLP	Multilayer Perceptron
NIDS	Network Intrusion Detection System
PGD	Projected Gradient Descent
RFC	Random Forest Classifier
ZOO	Zeroth-Order Optimization

References

1. Denning, D. An intrusion-detection model. *IEEE Trans. Softw. Eng.* **1987**, *SE-13*, 222–232. [CrossRef]
2. Almseidin, M.; Alzubi, M.; Kovacs, S.; Alkasassbeh, M. Evaluation of machine learning algorithms for intrusion detection system. In Proceedings of the 2017 IEEE 15th International Symposium on Intelligent Systems and Informatics, Subotica, Serbia, 14–16 September 2017; pp. 000277–000282.
3. Ahmad, Z.; Khanm, A.S.; Shiang, C.W.; Abdullah, J.; Ahmaad, F. Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Trans. Emerg. Telecommun. Technol.* **2021**, *32*, e4150. [CrossRef]
4. Corona, I.; Giacinto, G.; Roli, F. Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues. *Inf. Sci.* **2013**, *239*, 201–225. ISSN 0020-0255. [CrossRef]

5. Han, D.; Wang, Z.; Zhong, Y.; Chen, W.; Yang, J.; Lu, S.; Shi, X.; Yin, X. Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 2632–2647. [CrossRef]
6. Kumar, V.; Kumar, K.; Singh, M. Generating practical adversarial examples against learning-based network intrusion detection systems. *Ann. Telecommun.* **2024**. [CrossRef]
7. Rosenberg, I.; Shabtai, A.; Elovici, Y.; Rokach, L. Adversarial machine learning attacks and defense methods in the cyber security domain. *ACM Comput. Surv.* **2021**, *54*, 1–36. [CrossRef]
8. He, K.; Kim, D.D.; Asghar, M.R. Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Commun. Surv. Tutorials* **2023**, *25*, 538–566. [CrossRef]
9. Tan, S.; Zhong, X.; Tian, Z.; Dong, Q. Sneaking through security: Mutating live network traffic to evade learning-based NIDS. *IEEE Trans. Netw. Serv. Manag.* **2022**, *19*, 2295–2308. [CrossRef]
10. Sharon, Y.; Berend, D.; Liu, Y.; Shabtai, A.; Elovici, Y. TANTRA: Timing-based adversarial network traffic reshaping attack. *IEEE Trans. Inf. Forensics Secur.* **2021**, *17*, 3225–3237. [CrossRef]
11. Qiu, H.; Dong, T.; Zhang, T.; Lu, J.; Memmi, G.; Qiu, M. Adversarial attacks against network intrusion detection in IoT systems. *IEEE Internet Things J.* **2021**, *8*, 10327–10335. [CrossRef]
12. Szegedy, C.; Zaremba, W.; Sutskever, I.; Bruna, J.; Erhan, D.; Goodfellow, I.; Fergus, R. Intriguing properties of neural networks. In Proceedings of the International Conference on Learning Representations, Banff, AB, Canada, 14–16 April 2014.
13. Eykholt, K.; Evtimov, I.; Fernandes, E.; Li, B.; Rahmati, A.; Xiao, C.; Prakash, A.; Kohno, T.; Song, D. Robust Physical-World Attacks on Deep Learning Visual Classification. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Salt Lake City, UT, USA, 18–23 June 2018; pp. 1625–1634.
14. Geron, A. *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*, 3rd ed.; O'Reilly Media: Sebastopol, CA, USA, 2022.
15. Madry, A.; Makelov, A.; Schmidt, L.; Tsipras, D.; Vladu, A. Towards deep learning models resistant to adversarial attacks. In Proceedings of the International Conference on Learning Representations (ICLR), Vancouver, BC, Canada, 30 April–3 May 2018.
16. Chen, P.-Y.; Zhang, H.; Sharma, Y.; Yi, J.; Hsieh, C.-J. ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security, Dallas, TX, USA, 3 November 2017. . [CrossRef]
17. Brendel, W.; Rauber, J.; Bethge, M. Decision-based adversarial attacks: Reliable attacks against black-box machine learning models. In Proceedings of the International Conference on Learning Representations (ICLR), Vancouver, BC, Canada, 30 April–3 May 2018.
18. Chen, J.; Jordan, M.I.; Wainwright, M.J. HopSkipJumpAttack: A query-efficient decision-based attack. In Proceedings of the 2020 IEEE Symposium on Security and Privacy, San Francisco, CA, USA, 18–21 May 2020; pp. 1277–1294.
19. Carlini, N.; Wagner, D. Towards evaluating the robustness of neural networks. In Proceedings of the 2017 IEEE Symposium on Security and Privacy, San Jose, CA, USA, 22–26 May 2017.
20. Tavallaee, M.; Bagheri, E.; Lu, W.; Ghorbani, A. A detailed analysis of the KDD CUP 99 data set. In Proceedings of the Second IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), Ottawa, ON, Canada, 8–10 July 2009; pp. 1–6. [CrossRef]
21. Hettich, S. Kdd Cup 1999 Data, The UCI KDD Archive. Available online: https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html (accessed on 18 December 2024).
22. McHugh, J. Testing intrusion detection systems: A critique of the 1998 and 1999 darpa intrusion detection system evaluations as performed by lincoln laboratory. *ACM Trans. Inf. Syst. Secur. (TISSEC)* **2000**, *3*, 262–294. [CrossRef]
23. Mirsky, Y.; Doitshman, T.; Elovici, Y.; Shabtai, A. Kitsune: An ensemble of autoencoders for online network intrusion detection. In Proceedings of the Network and Distributed Systems Security Symposium (NDSS), San Diego, CA, USA, 18–21 February 2018.
24. Sharafaldin, I.; Lashkari, A.H.; Ghorbani, A.A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy—Volume 1: ICISSP, Madeira, Portugal, 22–24 January 2018; pp. 108–116. [CrossRef]
25. Alrayes, F.S.; Zakariah, M.; Amin, S.U.; Khan, Z.I.; Helal, M. Intrusion detection in IoT systems using denoising autoencoder. *IEEE Access* **2024**, *12*, 122401–122425. [CrossRef]
26. Wang, N.; Chen, Y.; Xiao, Y.; Hu, Y.; Lou, W.; Hou, Y.T. MANDA: On adversarial example detection for network intrusion detection system. *IEEE Trans. Dependable Secur. Comput.* **2023**, *20*, 1139–1153. [CrossRef]
27. Wang, Z. Deep learning-based intrusion detection with adversaries. *IEEE Access* **2018**, *6*, 38367–38384. [CrossRef] [PubMed]
28. Yang, K.; Liu, J.; Zhang, C.; Fang, Y. Adversarial examples against the deep learning based network intrusion detection systems. In Proceedings of the IEEE Military Communications Conference (MILCOM), Los Angeles, CA, USA, 29–31 October 2018; pp. 559–564. [CrossRef]
29. Goodfellow, I.J.; Shlens, J.; Szegedy, C. Explaining and harnessing adversarial examples. In Proceedings of the International Conference on Learning Representations, San Diego, CA, USA, 7–9 May 2015.
30. DMLC XGBoost, eXtreme Gradient Boosting, GitHub. Available online: https://github.com/dmlc/xgboost (accessed on 18 December 2024).
31. Scikit-Learn Library, Machine Learning in Python. Available online: https://scikit-learn.org/ (accessed on 18 December 2024).

32. Adversarial Robustness Toolbox, GitHub. Available online: https://github.com/Trusted-AI/adversarial-robustness-toolbox (accessed on 18 December 2024).
33. Papernot, N.; McDaniel, P.; Jha, S.; Fredrikson, M.; Celik, Z.B.; Swami, A. The limitations of deep learning in adversarial settings. In Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P), Saarbruecken, Germany, 21–24 March 2016.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.